Neurocomputing 574 (2024) 127279

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Spikeformer: Training high-performance spiking neural network with transformer

Yudong Li, Yunlin Lei, Xu Yang *

AETAS Lab., Beijing Institute of Technology, Beijing, 100081, China

ARTICLE INFO

Keywords:
Spiking neural networks
Transformer
Spatio-temporal attention
Deep neural architectures
Artificial intelligence

ABSTRACT

Although spiking neural networks (SNNs) have made great progress on both performance and efficiency over the last few years, their unique working pattern makes it hard to train high-performance low-latency SNNs and their development still lags behind traditional artificial neural networks (ANNs). To compensate this gap, many extraordinary works have been proposed, but these works are mainly based on the same network structure (i.e. CNN) and their performance is worse than their ANN counterparts, which limits the applications of SNNs. To this end, we propose a Transformer-based SNN, termed "Spikeformer", which outperforms its ANN counterpart on both static dataset and neuromorphic datasets. First, to deal with the problem of "data hungry" and the unstable training period exhibited in the vanilla model, we design the Convolutional Tokenizer (CT) module, which stabilizes training and improves the accuracy of the original model on DVS-Gesture by more than 16%. Besides, we integrate Spatio-Temporal Attention (STA) into Spikeformer to better incorporate the attention mechanism inside Transformer and the spatio-temporal information inherent to SNN. With our proposed method, we achieve 98.96%/75.89% top-1 accuracy on DVS-Gesture/ImageNet datasets with 16/4 simulation time steps. On DVS-CIFAR10, we further conduct energy consumption analysis and obtain 81.4%/80.3% top-1 accuracy with 4/1 time step(s), achieving 1.7/6.4× energy efficiency over its ANN counterpart. Moreover, our Spikeformer outperforms its ANN counterpart by 3.13% and 0.12% on DVS-Gesture and ImageNet respectively, indicating that Spikeformer may be a more suitable architecture for training SNNs compared to CNN. We believe that this work shall promote the development of SNNs to be in step with ANNs as much as possible. Code will be publicly available.

1. Introduction

Inspired by biological neural functionality, Spiking Neural Network (SNN) is known as a promising bionic model with low-power computation and has gained considerable attention in recent years. As the third generation of Artificial Neural Network (ANN), SNN is computationally more powerful than other neural networks with regard to the number of neurons that are needed [1]. When embedded on neuromorphic hardware such as TrueNorth [2] and Loihi [3], SNN shows great potential to process spatio-temporal information effectively with low energy consumption due to its intrinsic spatio-temporal characteristic and event-driven spike communication mechanism. Nevertheless, owing to the lack of appropriate learning algorithm, its performance is not as good as traditional neural networks.

To tackle this problem, there are two main routes to train a SNN with high performance. The first is the ANN-to-SNN method, which obtains a SNN by simulating a pre-trained ANN model's behavior [4–6]. With adequate simulation steps, the conversion method can achieve almost lossless accuracy compared to its ANN counterpart. However, extremely high latency is often required to achieve satisfying accuracy and limits the practical applications of SNNs. The other approach is the Surrogate Gradient (SG) method [7]. To handle the non-differentiability of the spiking mechanism, the SG method utilizes surrogate gradients to approximate the gradients of the non-differentiable activation function on the backpropagation process. This method can directly train SNNs with low latency, but it cannot achieve high performance comparable to leading ANNs. Overall, SNNs obtained by aforementioned methods suffer from either low performance or high latency which hinders the development of training large-scale SNN models.

To deal with it, there are several methods and models proposed to train large-scale SNNs, such as threshold-dependent batch normalization (tdBN) [8] and Spike-Element Wise (SEW) ResNet [9]. These methods and models are mainly based on Convolutional Neural Networks (CNNs), such as ResNet [10] and VGG [11], which dominate the

* Corresponding author.
E-mail address: pyro_yangxu@bit.edu.cn (X. Yang).

https://doi.org/10.1016/j.neucom.2024.127279
Received 26 September 2023; Received in revised form 25 December 2023; Accepted 14 January 2024
Available online 15 January 2024
0925-2312/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

field of computer vision for years. However, in the field of computer vision, CNNs are gradually taken place by Vision Transformers (ViT) because of their excellent capabilities at capturing long-range dependencies. With the emergence of Transformer [12] and ViT [13], they have been the dominant approaches in sequence modeling problems such as language modeling and video understanding. Actually, the tasks of SNNs (e.g. DVS-Gesture) can be viewed as sequence modeling problems as well. Hence, it is natural to integrate SNN with inherent spatio-temporal information into ViT with Spatio-Temporal Attention (STA). However, due to the "data hungry" property [13] and the substandard optimizability of ViT [14], the vanilla architecture suffers from poor performance and unstable training period. The ANN version obtains 82.29% on DVS-Gesture and the training accuracy even does not converge when integrated with spiking neurons.

To this end, we design the Convolutional Tokenizer (CT) module, which is a SNN-oriented convolutional block. With this module, we improve the top-1 accuracy of the original model on DVS-Gesture from 82.29% to 95.83%. And its SNN counterpart manages to converge and even obtains better result (i.e. 98.96% on DVS-Gesture). Furthermore, with STA, our method performs better in capturing spatial and temporal relationships and is more robust to simulation time steps compared to spatial-wise or temporal-wise attention. Even with extremely small time steps (i.e. 1 time step), we do not suffer from severe degradation (with only 1.1% accuracy loss on DVS-CIFAR10 compared to 4 time steps) and achieve 6.4× energy efficiency over its ANN counterpart (1.7× when given 4 time steps). Ultimately, on DVS-Gesture/DVS-CIFAR10, we obtain 98.96%/81.4% top-1 accuracy with 16/4 time steps. To further explore its potential for training large-scale SNNs, we also verify our method on the large-scale ImageNet dataset. Although SNNs are thought to be more suitable for processing neuromorphic data than static data, we still achieve 75.89% top-1 accuracy with 4 time steps, surpassing its ANN counterpart by 0.12%.

We summarize our contributions as follows:

- To deal with the "data hungry" property and substandard optimizability of ViT, we design the CT module and gain essential improvement of accuracy (over 16%) on DVS-Gesture.
- We integrate STA into SNN to better utilize the spatio-temporal information inside the spiking neurons and demonstrate its advantages over spatial-wise or temporal-wise attention.
- We propose a directly trained Transformer-based SNN, termed "Spikeformer", which may be an alternative scheme to CNN for training large-scale SNNs.
- We demonstrate the robustness of Spikeformer and achieve satisfying performance and efficiency on DVS-CIFAR10, DVS-Gesture, and ImageNet datasets.

2. Related work

2.1. Large-scale SNN model

In the past few years, as SNNs have attracted increasing attention, lots of extraordinary works on training large-scale SNN models have sprung up. [4] are the first to build a SNN deeper than 100 layers by conversion method. But it requires hundreds or thousands of time steps, which results in high latency. It is not until [8] propose the tdBN method that we can extend directly-trained SNNs from fewer than 10 layers to 50 layers. They effectively alleviate gradient vanishing or explosion and balance the threshold and input on each neuron to get appropriate firing rates with a modified batch normalization method. [9] further analyzes the gradient and identity map issues in SNNs, alter the connection of residual block, and proposes SEW ResNet, which extends the depth to 152 layers. These large-scale SNNs are mainly based on CNNs, while Transformer-based architectures have gained great success in various domains (e.g. NLP, CV).

2.2. Transformer

Since the proposal of Transformer [12], it has been a dominant approach in NLP due to its extraordinary capabilities at capturing long-range dependencies among words. [13] is the first to apply it in computer vision and achieves remarkable results, which sets off a wave of research on ViT. [13] notes that ViT lacks inductive biases inherent to CNN, and therefore it requires more data to gain superior results. This issue has subsequently led to a series of studies to follow [15–19]. [18] introduces inductive biases into ViT with a convolutional head and successfully alleviates "data hungry". Moreover, [14] proposes that ViT exhibits substandard optimizability due to the patchify stem of ViT. They empirically demonstrate that early convolution can increase optimization stability of ViT and also improve peak performance. Despite the rapid development of Transformers in the field of ANNs, they are rarely used in SNNs due to the "data hungry" property and the substandard optimizability. Based on these observations, we propose the CT module and successfully train a high-performance Transformer-based SNN with low latency. Prior to this work, there are a few works applying Transformer to SNNs [20,21]. [20] converts a pre-trained Transformer to SNN, but this method requires relatively long time steps and has limited usages. [21] combines SNN and Transformer in a parallel way, which are decoupled and the Transformer can be replaced by other backbones. Their proposed Spiking Transformers are targeted at a specific task, while we are studying a more general architecture. [22] explores a Transformer-based SNN that is orthogonal to ours. We study the cruciality of early convolution and the incorporation of spatio-temporal attention and SNNs with inherent spatio-temporal characteristics, while [22] focus on spike-based attention. Our method can be seamlessly incorporated with the method proposed by [22].

2.3. Attention mechanism in SNN

The attention mechanism enables the model to pay more attention to the most informative components of input. It has been applied in SNN as spatial-wise attention [23–25] or temporal-wise attention [26]. However, these works have limits, since they either capture dependencies inside frames or utilize the statistical characteristics of the frames input at different time steps, which may lose information in spatial or temporal domain. Moreover, spiking neurons inherently contain rich spatio-temporal information due to their neural dynamics, so we integrate both spatial and temporal attention into SNN to better utilize SNNs' spatio-temporal characteristics. [21] also adopts STA, but their STA does not have interaction across temporal domain and their "temporal" attention is used to enhance the temporal features from SNN branch and does not consider temporal relationships, while ours interacts multiple times taking direct spatio-temporal relationships into consideration.

3. Method

3.1. Spiking neuron model

In general, spiking neurons can be classified into two classes by their data transmission form. One is the spike-based neuron, such as IF, LIF [27], and PLIF [28], which uses spike streams for communication. When embedded on neuromorphic hardware, this kind of neurons will skip computation if they have not received any input spike, i.e. event-driven computation. Thus they can process spatio-temporal information in an energy-saving way. The other is the analog-based neuron, such as LIAF [29], which uses the dynamic characteristics of spiking neurons but transmits analog values. This kind of neurons possesses strong representational capabilities due to their inherent spatio-temporal feature, but they cannot skip computation, which makes them cost more energy than spike-based neurons. Without loss of generality, we experiment

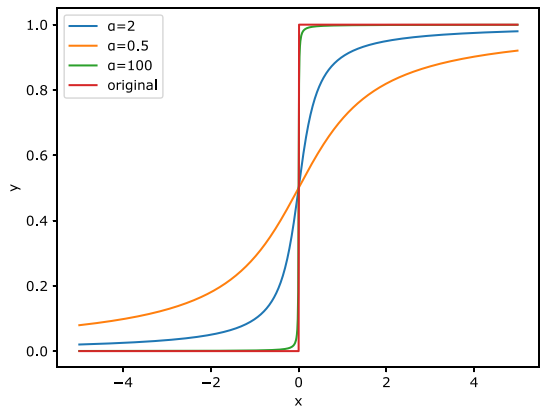


Fig. 1. Visualization of the surrogate function and the original activation function. The higher the α, the steeper the surrogate function. We set $\alpha = 2$ for all experiments.

with both types of neurons. Formally, We use the following equations to describe the dynamics of spiking neurons:

$$H[t] = \begin{cases} V[t-1] + \frac{1}{\tau}(X[t] - (V[t-1] - V_{reset})), & LIF \\ V[t-1] + X[t], & IF \end{cases} \tag{1}$$

$$S[t] = \Theta(H[t] - V_{th}), \tag{2}$$

$$V[t] = H[t](1 - S[t]) + V_{reset}S[t], \tag{3}$$

where $H[t]$ denotes the membrane potential after integrating input at time step t, while $V[t]$ denotes the membrane potential after the trigger of a spike at time step t. $X[t]$ is the input at time-step t, and $S[t]$ is the output spike at time-step t. The input propagates to the next layer is $S[t]$ for IF/LIF model and $ReLU(H[t])$ for LIAF model. $\Theta(x)$ is the Heaviside step function, which is defined by $\Theta(x) = 1$ for $x > 0$, otherwise $\Theta(x) = 0$. V_{th} and V_{reset} denote firing threshold and reset potential, respectively. τ represents the membrane time constant. When τ is learnable, we have PLIF model [28] and PLIAF model. Note that the time steps that we adopt here to simulate the behavior of spiking neurons are the time steps that we adopt in Section 4. When the model only demands a few time steps to obtain satisfying results, we term it as a low-latency model.

For all experiments, we adopt the same surrogate gradient function to approximate the gradients of the non-differentiable activation function on the backpropagation process. The function can be formulated as follows:

$$\sigma(x) = \frac{1}{\pi}\arctan(\frac{\pi}{2}\alpha x) + \frac{1}{2}, \tag{4}$$

$$\sigma'(x) = \frac{\alpha}{2(1 + (\frac{\pi}{2}\alpha x)^2)}, \tag{5}$$

where α is the slope parameter controlling how steep the function is. This parameter can be further set as a learnable parameter, but this is out of the scope of this paper. We set $\alpha = 2$, $V_{reset} = 0$ and $V_{th} = 1$ for all neurons. We detach $S[t]$ in Eq. (2) in the backward computational graph to improve performance [30]. The original activation function and surrogate gradient function are visualized in Fig. 1. Note that with the increase of the value of α, the surrogate function becomes steeper. Softer surrogate function provides more rough gradients and steeper function is easier to cause gradient vanishing. We adopt $\alpha = 2$ to achieve a trade-off. The activation function works fine during the forward propagation, but the surrogate function takes its place in the computational graph. And thus, we manage to backpropagate the gradients of the non-differentiable activation function.

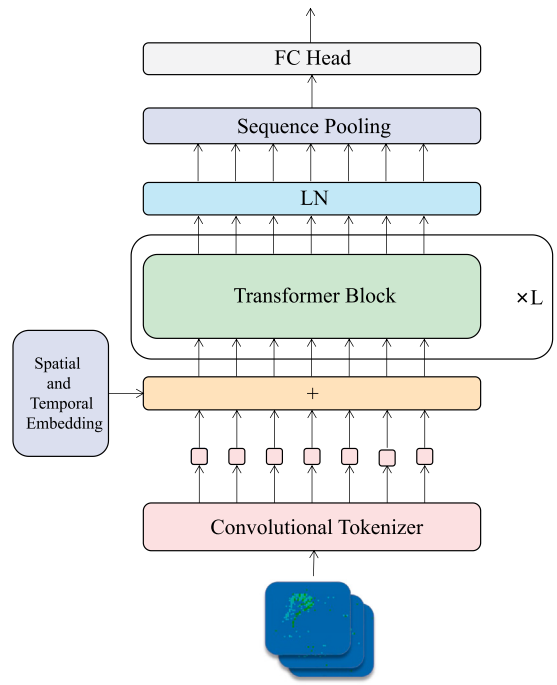


Fig. 2. An overview of Spikeformer. The images are processed by CT module first and then flattened into a sequence of token embeddings. The tokens are positional encoded and feed into transformer blocks with STA. Lastly, the outputs from the last transformer block are pooled and sent to the classification head.

3.2. Model architecture

An overview of Spikeformer is depicted in Fig. 2. Compared to the standard ViT, we use a Convolutional Tokenizer (CT) module to process a series of 2D frames and then reshape them into a sequence of flattened token embeddings instead of the original patchify stem. Furthermore, to better utilize the spatio-temporal information and achieve a trade-off between accuracy and computational complexity, we adopt divided Spatio-Temporal Attention [31] in Transformer block. Finally, as it is non-trivial to deal with class token in divided Spatio-Temporal Attention and the class token may lose information when discarding embeddings [32], we pool the sequential based information from the last transformer block using a sequential pooling method [18], which may be a better choice than global average-pooling as it can learn to assign more weights to more informative tokens. We would elaborate each component in the following subsections.

3.2.1. Convolutional Tokenizer

In order to alleviate "data hungry" and help stabilize the training period, we propose the CT module to introduce inductive biases into Spikeformer. The architecture of CT module is illustrated in Fig. 3. Given a series of input frames $x \in \mathbb{R}^{T\times H\times W\times C}$:

$$Conv(x) = SN(BN(Conv2d(x))), \tag{6}$$

$$CT_d(x) = Conv(Conv(x)) + Conv(AvgPool(x)), \tag{7}$$

$$CT(x) = Conv(Conv(x)) + x, \tag{8}$$

where $Conv2d(\cdot)$ and $BN(\cdot)$ denote convolutional layers and batch normalization layers, which will process each time step separately, and $SN(\cdot)$ denotes spiking neurons that will integrate input and output spike trains. $CT_d(\cdot)$ and $CT(\cdot)$ depict downsample module and normal module respectively. Multiple downsample modules and normal modules are stacked together to encode input frames and output spike trains to the following blocks.

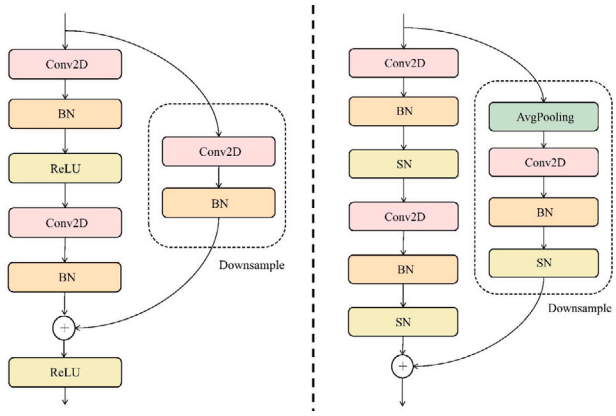


Fig. 3. Comparison of the original block in ResNet and the block in CT module. Note that the part enclosed by the dotted line only appears when downsampling. The left is the block in ResNet and the right is the block in CT module.

The design of the CT module mainly follows the architecture of SEW block [9] as it is proven to implement identity mapping and is beneficial for training deep spiking neural networks. But in the downsample module, we use a stride-2 2×2 average pooling layer and a stride-1 Conv1 $\times$ 1 instead of a stride-2 Conv1 $\times$ 1 in the shortcut connection in order to not discard information [33]. The comparison of the original block in ResNet [10] and the block in our CT module is shown in Fig. 3. We formulate the basic block without downsampling as follows:

$$X^{l+1} = ReLU(F^l(X^l) + X^l), \tag{9}$$

$$X^{l+1} = SN(F^l(X^l) + X^l), \tag{10}$$

where X^l and $F^l(\cdot)$ denote the input and the residual mapping in the lth block. Eqs. (9) and (10) are the ANN version and SNN version respectively. Assuming that the blocks are zero-initialized (i.e. all weights equal to zero), we have:

$$X^{l+1} = ReLU(X^l), \tag{11}$$

$$X^{l+1} = SN(X^l). \tag{12}$$

In Eq. (11), X^l has been processed by the $ReLU$ function of the $(l-1)^{th}$ block, and thus $X^{l+1} = X^l$, which is identity mapping. As for Eq. (12), due to the leaky property of most spiking neurons, it is hard to implement identity mapping unless we adopt IF neuron model and assign a small enough value to V_{th}, which enables the neurons to output a spike as soon as they receive a spike. But this compromise limits the applications of SNNs, so the original design of ResNet is not suitable for SNNs. To deal with it, we modify its topology and formulate it as follows:

$$X^{l+1} = SN(F^l(X^l)) + X^l. \tag{13}$$

and with this we could achieve identity mapping (e.g. $F^l(\cdot)$ always outputs 0).

Spikes are propagating through these blocks. With zero-initialized, it can easily implement identity mapping. Because the $ReLU$ function will not shrink input, $X^{l+1} = ReLU(F^l(X^l)) + X^l \geq X^l$, which will keep accumulating and cause infinite outputs in deep layers. That is to say, this architecture does not fit into its ANN counterpart. But the output of spiking neurons is 0/1, so every time there is a downsample block, the input will be no greater than 2, which effectively tackles this issue.

3.2.2. Transformer block

Our Spikeformer contains L encoding blocks and the architecture of our transformer block is shown in Fig. 4. The output of the last CT module $\hat{x} \in \mathbb{R}^{T \times H' \times W' \times D}$ is flattened into vectors $\hat{a}_{(p,t)} \in \mathbb{R}^D$,

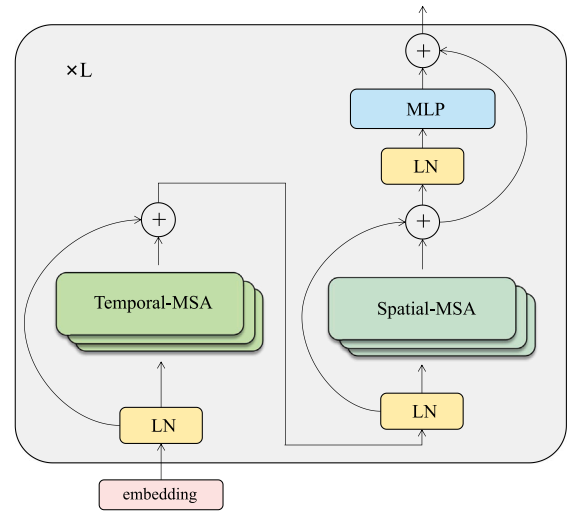


Fig. 4. Illustration of Transformer Block. The transformer block first conduct temporal attention and then spatial attention. Each submodule is preceded by a LN layer.

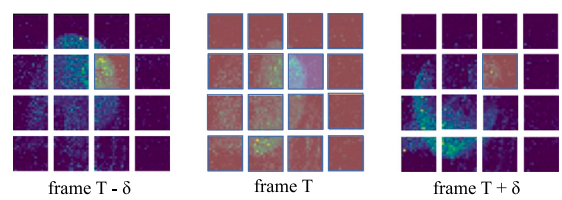


Fig. 5. Illustration of Divided Spatio-Temporal Attention scheme. We denote in white the query patch and show in orange its self-attention space–time neighborhood. Patches without color are not used for self-attention computation of the white patch. Note that δ is an arbitrary integer that enables T$\pm\delta$ to cover all images (i.e. temporal attention has a global receptive field). Frame T$-\delta$ and frame T$+\delta$ denote the frame before time step T and after time step T, respectively.

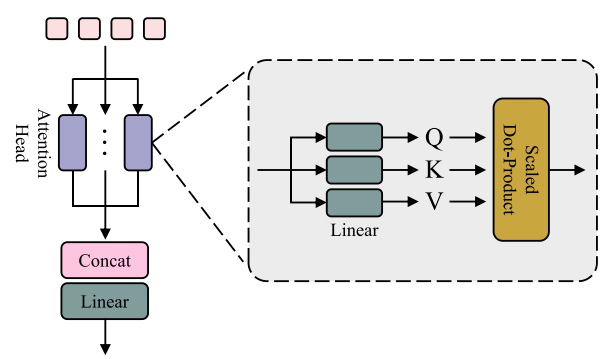


Fig. 6. Illustration of Multi-head Self Attention. Tokens for spatial attention and temporal attention are selected according to Fig. 5 and fed to each attention head separately. The results from each head are concatenated together and processed by a Linear layer.

with $p = 1, \ldots, N$ denoting spatial locations and $t = 1, \ldots, T$ denoting temporal locations. $N = H' \times W'$, and D depicts the dimensionality of tokens. To encode the spatio-temporal position of each token, we add a learnable positional embedding $e^{pos}_{(p,t)} \in \mathbb{R}^D$ to each token:

$$z^{(0)}_{(p,t)} = \hat{a}_{(p,t)} + e^{pos}_{(p,t)} \tag{14}$$

Each encoding block l will compute a query/key/value vector:

$$q^{(l,h)}_{(p,t)} = LN(z^{(l)}_{(p,t)}) W^{(l,h)}_Q \in \mathbb{R}^{D_h} \tag{15}$$

$$k^{(l,h)}_{(p,t)} = LN(z^{(l)}_{(p,t)}) W^{(l,h)}_K \in \mathbb{R}^{D_h} \tag{16}$$

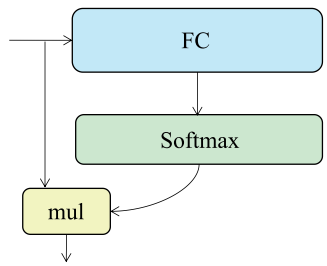


Fig. 7. Illustration of Sequence Pooling. FC and Softmax are meant to provide a learnable weight to the sequence of tokens from the last transformer block and aggregate features for the final classification head.

$$v_{(p,t)}^{(l,h)} = LN(z_{(p,t)}^{(l)})W_V^{(l,h)} \in \mathbb{R}^{D_h} \tag{17}$$

where $LN(\cdot)$ denotes LayerNorm [34], $h = 1, \dots, H$ is the index over multiple attention heads and $D_h = D/H$ is the dimensionality of each query/key/value vector. In Spikeformer, the attention mechanism is divided into spatial attention and temporal attention:

$$h_{(p,t)}^{(l,h)space} = \sum_{p'=1}^{N} SM(\frac{q_{(p,t)}^{(l,h)^T}}{\sqrt{D_h}}\left[\{k_{(p',t)}^{(l,h)}\}_{p'=1,\dots,N}\right])v_{(p',t)}^{(l,h)}, \tag{18}$$

$$h_{(p,t)}^{(l,h)time} = \sum_{t'=1}^{T} SM(\frac{q_{(p,t)}^{(l,h)^T}}{\sqrt{D_h}}\left[\{k_{(p,t')}^{(l,h)}\}_{t'=1,\dots,T}\right])v_{(p,t')}^{(l,h)}, \tag{19}$$

$$s_{(p,t)}^{(l)} = Concat(h_{(p,t)}^{(l,1)}, \dots, h_{(p,t)}^{(l,H)}) \cdot W_O, \tag{20}$$

where $SM(\cdot)$ denotes the softmax activation function, $W_O \in \mathbb{R}^{H \cdot D_h \times D}$ is the projection matrix, and s is the output of temporal multi-head self attention (TMSA) or spatial multi-head self attention (SMSA). Thus a transformer block in Spikeformer can be formulated as:

$$z_l' = TMSA(LN(z_{l-1})) + z_{l-1} \tag{21}$$

$$z_l'' = SMSA(LN(z_l')) + z_l' \tag{22}$$

$$z_l = FC(SN(FC(LN(z_l'')))) + z_l'' \tag{23}$$

with $l = 1, \dots, L$ depicting an index over blocks.

The divided Spatio-Temporal attention (STA) can be illustrated by Fig. 5 intuitively and the structure of multi-head self attention (MSA) is shown in Fig. 6. Note that the divided STA in Fig. 5 is intended for selecting query/key/value tokens that will be adopted by MSA in Fig. 6. Both TMSA and SMSA are depicted in Fig. 6, and the only difference is the selected tokens. The query token in frame T first computes its attention scores with other tokens that are at the same location in other frames as temporal attention. And then it computes its attention scores with other tokens that are in the same frame as spatial attention. Not only can this scheme reduce memory cost and computational complexity, but it also achieves satisfying accuracy.

3.2.3. Sequence pooling

In order to fully utilize the information in the outputs of the last transformer block, instead of discarding most of them (i.e. class token), we adopt Sequence Pooling (SP) to compute the weighted sum of the outputs and send the result to the classifier, as shown in Fig. 7. Given the normalized output tensor of the last transformer block $z^l \in \mathbb{R}^{T \times N \times D}$, we feed z^l to a fully connected (FC) layer, apply softmax activation, and transpose it:

$$W = SM(FC(z^l))^T. \tag{24}$$

The FC layer is equivalent to a matrix $W_{FC} \in \mathbb{R}^{D \times 1}$ and thus $W \in \mathbb{R}^{T \times 1 \times N}$. Then we multiply z^l by W:

$$\hat{z} = W \times z^l \in \mathbb{R}^{T \times 1 \times D}, \tag{25}$$

squeeze the second dimension and get $z \in \mathbb{R}^{T \times D}$. Ultimately, we send the weighted sum $\hat{z}$ to the classifier with a minimal loss of information.

3.2.4. Loss function

The aggregated feature $z_t \in \mathbb{R}^D$ at each time step t is fed to the final classification head separately. And the outputs from the classifier are average across all time steps to get the final result:

$$y_t = FC(z_t) \in \mathbb{R}^C, \tag{26}$$

$$y = \frac{1}{T}\sum_{t=1}^{T} y_t \in \mathbb{R}^C, \tag{27}$$

where y_t, y, C are the output at time step t, averaged output, and the number of classes, respectively. Then the outputs are processed by a softmax layer and the loss function is determined as the cross-entropy:

$$p = Softmax(y) \in \mathbb{R}^C, \tag{28}$$

$$L = -\sum_{i=1}^{C} \hat{y}_i log(p_i), \tag{29}$$

where p, L, $\hat{y}_i$, and p_i denote the probability of each class, final loss, class label for class i (1 if it belongs to class i, else 0), and the probability of belonging to class i. We backpropagate the gradients of the loss L on both spatial and temporal domains by backpropagation through time (BPTT) as did in [8].

4. Experiment

We evaluate our models on static dataset (ImageNet) and neuromorphic datasets (DVS-CIFAR10, DVS-Gesture). Extensive ablation experiments reveal the cruciality of CT module and the robustness of our method in terms of depth and time steps. And then we compare our results with state-of-the-art methods and some concurrent works to demonstrate the superiority of our method. Finally, we compare the energy consumption of our method and its ANN counterpart on DVS-CIFAR10 to explore its energy-efficient potential. Note that the ANN counterparts have the same architecture as their SNN versions, except that the topology of early convolutional layers is original residual connection instead of SEW ResNet and the spiking neurons are replaced by ReLU activation function.

4.1. Experimental setup

4.1.1. DVS-Gesture

DVS-Gesture [35] contains 1342 records from 23 subjects in the training set and 288 examples from 6 subjects for testing captured by DVS cameras under three lighting conditions, belonging to 11 classes. The input resolution of DVS-Gesture is 128 × 128. We use the same AER data pre-processing method as [28] and do not adopt any data augmentation for this dataset. We train our model using Adam [36] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, a batch size of 16, and apply a weight decay of 0.00015. We warm up for 20 epochs and train the model for a total of 150 epochs with a learning rate of 0.001.

4.1.2. DVS-CIFAR10

DVS-CIFAR10 [37] is converted from the famous CIFAR10 dataset to its dynamic form by scanning each sample in front of DVS cameras. It consists of 1000 images per class with total 10 classes. The training set contains 9000 images and the testing set contains 1000 images. The input resolution and the AER data pre-processing method of DVS-CIFAR10 are the same as DVS-Gesture. Because DVS-CIFAR10 is more challenging than DVS-Gesture, we adopt additional data augmentation. First, we horizontally flip the input frames with a possibility of 0.5. Then we sample two values a and b from an integer uniform distribution $U(-5, 5)$, horizontally move the input frames for a pixels, and vertically move the input frames for b pixels. When a/b is positive, the frames

Table 1
Architectures of the CT module. Note that the blocks here are convolutional blocks. (5 × 5, 32) denotes the convolution layer with 32 (5 × 5) kernels. And each block will downsample the input with a stride of 2.

	Spikeformer-4/ 5×1×3	Spikeformer-7/ 5×1×3	Spikeformer-7/ 3×2×3	Spikeformer-7L/ 3×2×3
block1	$\begin{pmatrix} 5\times5, 32 \\ 5\times5, 32 \end{pmatrix} \times 1$	$\begin{pmatrix} 5\times5, 64 \\ 5\times5, 64 \end{pmatrix} \times 1$	$\begin{pmatrix} 3\times3, 64 \\ 3\times3, 64 \end{pmatrix} \times 2$	$\begin{pmatrix} 3\times3, 64 \\ 3\times3, 64 \end{pmatrix} \times 2$
block2	$\begin{pmatrix} 5\times5, 64 \\ 5\times5, 64 \end{pmatrix} \times 1$	$\begin{pmatrix} 5\times5, 128 \\ 5\times5, 128 \end{pmatrix} \times 1$	$\begin{pmatrix} 3\times3, 128 \\ 3\times3, 128 \end{pmatrix} \times 2$	$\begin{pmatrix} 3\times3, 128 \\ 3\times3, 128 \end{pmatrix} \times 2$
block3	$\begin{pmatrix} 5\times5, 128 \\ 5\times5, 128 \end{pmatrix} \times 1$	$\begin{pmatrix} 5\times5, 256 \\ 5\times5, 256 \end{pmatrix} \times 1$	$\begin{pmatrix} 3\times3, 256 \\ 3\times3, 256 \end{pmatrix} \times 2$	$\begin{pmatrix} 3\times3, 256 \\ 3\times3, 256 \end{pmatrix} \times 2$
block4				$\begin{pmatrix} 3\times3, 512 \\ 3\times3, 512 \end{pmatrix} \times 2$

Table 2
Details of Spikeformer model variants. #Layer denotes the number of transformer block (i.e. depth). #Heads denotes the number of head in multi-head self-attention. Ratio denotes the MLP expansion ratio. Dim denotes the dimension of the transformer blocks.

Model	#Layers	#Heads	Ratio	Dim
Spikeformer-2	2	2	1	128
Spikeformer-4	4	2	1	128
Spikeformer-7	7	4	2	256
Spikeformer-10	10	4	2	256
Spikeformer-7L	7	8	3	512
Spikeformer-14	14	6	3	384

Table 3
Ablation studies on the Convolutional Tokenizer (CT) module, spiking neurons (SN), and the Sequence Pooling (SP) module on DVS-Gesture dataset. N/A denotes that the model does not converge. CT module manages to effectively alleviate "data hungry" issue in vanilla design and the integration of SN and SP also enhances the performance.

CT	SN	SP	Accuracy (%)
✗	✗	✔	82.29
✗	✔	✔	N/A
✔	✗	✔	95.83
✔	✔	✗	97.12
✔	✔	✔	**98.96**

Table 4
Ablation on the CT architecture. Note that increasing the number of tokens to 4096 causes a GPU memory overflow. When there are about 200 tokens, Spikeformer performs best on both DVS-CIFAR10 and DVS-Gesture. And we determine the number of downsample blocks in CT module based on the "200 tokens rule". "SPS" denotes the design of early convolution in [22].

Dataset	Tokens	Accuracy (%)	
		Ours	SPS
DVS-CIFAR10	4096	N/A	N/A
	1024	73.1	71.3
	256	**80.0**	77.4
	64	79.6	76.8
	16	75.4	72.1
	4	73.0	69.7
DVS-Gesture	4096	N/A	N/A
	1024	82.64	79.31
	256	**90.97**	87.33
	64	88.89	85.12
	16	87.50	83.89
	4	85.42	81.21

are moved upwards/rightwards, and vice versa. Finally, we randomly select two values l and h from an integer uniform distribution $U(1, 16)$ as length and height, mask off an area of the frames with the selected l, and w, and adopt label-smoothing regularization [38] with the hyperparameter set to 0.14. We train our model using Adam [36] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, a batch size of 32, and apply a weight decay of 0.0001. We warm up for 25 epochs and train the model for a total of 600 epochs with an initial learning rate of 0.001. Every 192 epochs, we decay the learning rate from lr to $0.1lr$.

4.1.3. ImageNet

ImageNet [39] is the most popular benchmark dataset used for large-scale image classification, containing 1.28 million training images and 50,000 validating images. We first randomly crop the images to 224 × 224 and horizontally flip the images with a possibility of 0.5. Auto-Augment [40] and label-smoothing regularization with the hyperparameter set to 0.1 are adopted for further augmentation. We train our model using SGD with a momentum of 0.9, a total batch size of 160 on 8 GPUs, and apply a weight decay of 0.0001. We warm up for 5 epochs and train the model for a total of 130 epochs with an initial learning rate of 0.01 and a cosine learning rate decay scheduler [41]. Besides, we manually decay the learning rate by 0.1 when the training accuracy tends to plateau (at epoch {94, 115, 118} respectively).

We adopt mixed precision training [42] to accelerate training and save memory. For ImageNet, we train our model on multi-GPU and adopt sync-BN technique [43] to reduce the influence of small batch size. Additionally, we adopt Droppath [44] regularization with hyperparameter set to 0.1 for all experiments. For experiments on DVS-Gesture and DVS-CIFAR10, we conduct experiments with 1 Nvidia RTX 3090 GPU, while on ImageNet, we use 8 Nvidia RTX 3090 GPUs. Our implementations are based on PyTorch [45] and SpikingJelly [46]. Our code will be publicly available, and for reproducibility, we use the identical seed in all experiments and will detail our configurations in our code repository.

4.2. Model variants

Our Spikeformer configurations are summarized in Table 2. We use brief notation to indicate model variants and the CT stem: for instance, Spikeformer-7/5 × 2 × 3 means the Spikeformer-7 variant using $2 \times 3 = 6$ CT modules with kernel size 5 and every 2 CT modules have 1 downsample module. Note that every downsample CT module will reduce the number of tokens by 4, which will be further discussed in the ablation studies of CT module. The detailed architectures of CT module are shown in Table 1. The first convolutional layer of each block will downsample the input with a stride of 2 and each block will double the dimension until it reaches the dimension of transformer block. (5 × 5, 32) denotes the convolution layer with 32 (5 × 5) kernels.

4.3. Ablation studies

We perform extensive ablation experiments to verify the importance of the CT module and the Sequence Pooling (SP) module and draw a empirical rule to design the CT module. Besides, since [22] also adopt early convolution, but with a design similar to [18], we compare the effect of different designs and demonstrate that ours is more suitable for SNNs. And then we conduct experiments with various time steps and layers of transformer block to further demonstrate the robustness of our method. Note that the ablation studies are not aimed at pushing SOTA results, so we adopt relatively simple training recipes.

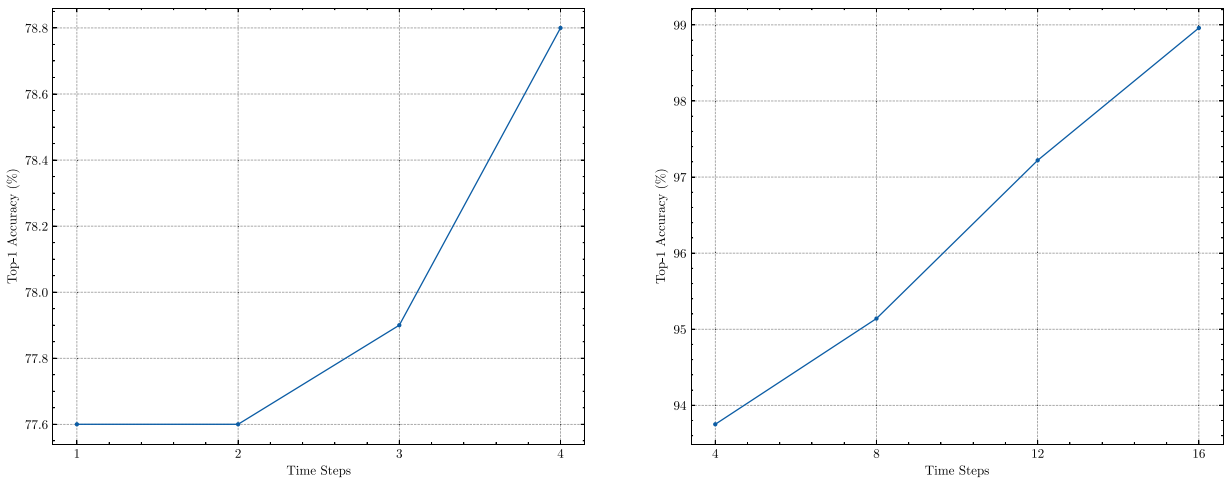


Fig. 8. Accuracy curve of various time steps on DVS-Gesture and DVS-CIFAR10. The left is the curve on DVS-CIFAR10 and the right is the curve on DVS-Gesture.

Table 5
Ablation on the depth of the transformer blocks. Note that depth denotes the number of transformer block (e.g. Spikeformer-4 has 4 transformer blocks). The performance consistently increases as the network deepens on both DVS-Gesture and DVS-CIFAR10.

Dataset	Model	Acc. (%)
DVS-Gesture	Spikeformer-14/5 × 1 × 3	97.22
	Spikeformer-10/5 × 1 × 3	97.96
	Spikeformer-7/5 × 1 × 3	**98.96**
	Spikeformer-4/5 × 1 × 3	97.22
	Spikeformer-2/5 × 1 × 3	96.88
DVS-CIFAR10	Spikeformer-14/3 × 2 × 3	78.0
	Spikeformer-10/3 × 2 × 3	79.4
	Spikeformer-7/3 × 2 × 3	**80.0**
	Spikeformer-4/3 × 2 × 3	78.6
	Spikeformer-2/3 × 2 × 3	77.9

Table 6
Ablation on time steps and attention mechanisms. Note that time steps indicate the times that the state of spiking neurons has updated (i.e. simulation time steps). STA performs best among these attention mechanisms. Temporal attention performs better on DVS-Gesture than spatial attention, while on DVS-CIFAR10, spatial attention performs better.

Dataset	T	Spatial	Temporal	STA
DVS-Gesture	4	91.76	92.84	93.75
	8	93.07	94.22	95.14
	12	94.85	96.43	97.22
	16	96.54	**97.66**	**98.96**
DVS-CIFAR10	1	73.2	72.7	77.6
	2	73.6	72.9	77.7
	3	74.3	73.2	77.9
	4	**76.4**	74.1	**78.8**

4.3.1. SP and CT module

When using the patchify stem, the ANN version performs poorly (82.29%) on DVS-Gesture. There is a large gap between the training accuracy (100%) and the testing accuracy (82.29%). The model is severely overfitted because of the lack of inductive biases. And the training accuracy of the SNN version even does not converge because of the substandard optimizability caused by the patchify stem [14]. To deal with it, we propose CT module to help stabilize the training period, alleviate "data hungry" and make the model generalize better. In Table 3, we can observe that CT module greatly boosts the accuracy. With CT module, we obtain over 10% improvement for ANN version and successfully train the SNN version (i.e. Spikeformer), which further enlarges the gap to more than 16%. Note that the integration of SP module also brings 1.84% improvement on DVS-Gesture. These results illustrate the effectiveness of the CT module and SP module.

Moreover, the design of the CT module is crucial for the performance of Spikeformer. The more the downsample blocks, the fewer the number of tokens. And the number of tokens determines the granularity of the feature map, which will influence the performance of transformer blocks [13]. From Table 4, we note that Spikeformer performs best with around 200 tokens. Thus, we follow this rule to design our models in the following experiments. Besides, the SPS in [22] share a similar design of early convolution with [18], while we adopt a SNN-oriented design, which shows better performance in Table 4.

4.3.2. Depth and time steps

In addition to the CT module, we also conduct depth analysis on transformer blocks. Note that the depth denotes the number of transformer blocks (e.g. Spikeformer-4 has 4 transformer blocks). We experiment with five model variants on DVS-Gesture and DVS-CIFAR10 datasets, as shown in Table 5. Table 5 shows that even with shallow architecture (i.e. Spikeformer-2), our model still performs well. And the accuracy on both datasets consistently increases with the deepening of transformer blocks until it reach a depth of 7. Based on these results, we apply Spikeformer-7L to ImageNet and adopt Spikeformer-7 as our default setting.

To further demonstrate the robustness of our method, we evaluate our model with different time steps, as shown in Table 6 and Fig. 8. On DVS-CIFAR10 dataset, our method does not suffer from severe degradation even with extremely low time steps. We attribute this phenomenon to the dataset itself. DVS-CIFAR10 is collected by scanning static images with DVS cameras. Hence, this dataset inherently does not contain much temporal information. The performance on DVS-CIFAR10 is more relevant to the ability to capture spatial relationships. Rather, DVS-Gesture dataset contains abundant temporal information because it is a collection of moving gestures performed by different individuals. Thus, its performance is more sensitive to the simulation time steps. The performance on DVS-Gesture is more relevant to the ability to capture temporal relationships.

To further demonstrate these conjectures and the effectiveness of STA, we also compare the performance of spatial-wise attention and temporal-wise attention in Table 6. As we analyzed, spatial attention performs better on DVS-CIFAR10 but suffers from severe degradation on DVS-Gesture with small time steps. Temporal attention is robust to simulation time steps but fails to obtain satisfying accuracy on DVS-CIFAR10. And our method achieves superior results on both datasets with various time steps, which demonstrates the necessity of incorporating STA into SNNs.

Table 7
Comparison with the SOTA methods on DVS-Gesture dataset.

Work	Model	Time Steps	Neuron	Accuracy (%)
Shen et al. 2022[47]	ResNet-18	10	PLIF	96.75
Zheng et al. 2021[8]	ResNet-17	40	LIF	96.87
Fang et al. 2021[9]	7B-Net	16	PLIF	97.92
Fang et al. 2021[28]	5Conv, 2Fc	20	PLIF	97.57
Yao et al. 2021[26]	TA-SNN	60	LIF	95.48
			LIAF	98.61
This work	Spikeformer-7/5 × 1 × 3	**10**	LIF	96.89
		16		**98.96**

Table 8
Comparison with the SOTA methods on DVS-CIFAR10 dataset.

Work	Model	Time Steps	Neuron	Parameters(M)	Accuracy (%)
Zheng et al. 2021[8]	ResNet-19	10	LIF	14.66	67.8
Fang et al. 2021[9]	Wide-7B-Net	4, 8, 16	PLIF	**1.19**	64.8, 70.2, 74.4
Fang et al. 2021[28]	4Conv, 2Fc	20	PLIF	17.4	74.8
Yao et al. 2021[26]	TA-SNN	10	LIF	2.10	71.1
			LIAF	2.10	72.0
Li et al. 2021[48]	ResNet-18	10	LIF	11.69	75.4
Meng et al. 2022[49]	VGG-11	20	LIF	132.86	77.3
Shen et al. 2022[47]	ResNet-18	10	PLIF	11.69	81.45
Li et al. 2022[50]	VGG-11	10	LIF	132.86	81.7
This work	Spikeformer-7/3 × 2 × 3	**1**	PLIF	9.28	**80.3**
		4			**81.4**

Table 9
Comparison with the SOTA methods on ImageNet dataset.

Method	Work	Model	T	Par.(M)	Accuracy (%)
ANN	This work	ANN counterpart	–	38.75	75.77
ANN-to-SNN	Sengupta et al. 2019[6]	VGG-16	2500	138.36	69.96
	Hu et al. 2018[4]	ResNet-50	350	25.56	72.75
	Han et al. 2020[5]	VGG-16	4096	138.36	73.09
Directly training	Zheng et al. 2021[8]	ResNet-34	6	21.80	63.72
		ResNet-50		25.56	64.88
		ResNet-34(large)		86.13	67.05
	Fang et al. 2021[9]	ResNet-34	4	21.80	67.04
		ResNet-50		25.56	67.78
		ResNet-101		44.55	68.76
		ResNet-152		60.19	69.26
	Li et al. 2021[48]	ResNet-34	6	21.80	68.19
		VGG-16	5	138.36	71.24
	Meng et al. 2022[49]	PreAct-ResNet-18	50	**11.69**	67.74
	This work	Spikeformer-7L/3 × 2 × 4	**4**	38.75	**75.89**
					78.31[a]

[a] Denotes that we adopt PLIAF model.

4.4. Comparison with SOTA methods

Since the input resolution of DVS-Gesture and DVS-CIFAR10 is 128 × 128, we downsample the input frames for 3 times, which results in 256 tokens. And for ImageNet with an input size of 224 × 224, we downsample the input for 4 times, which results in 196 tokens. Because DVS-Gesture dataset contains rich temporal information, it needs more time steps to gain better performance and takes up more memory. To balance the memory cost, we adopt Spikeformer-7/5 × 1 × 3 for DVS-Gesture and Spikeformer-7/3 × 2 × 3 for DVS-CIFAR10. But the original Spikeformer-7 variant is too simple for ImageNet dataset, we adopt Spikeformer-7L/3 × 2 × 4 for ImageNet and compare these methods with SOTA methods and concurrent works.

4.4.1. DVS-Gesture

In Table 7, we compare our method with SOTA methods on DVS-Gesture. With less than half of their simulation time steps, we outperform [8] by 2.09%. Also with less time steps, we outperform [28] by 1.39%. [9] achieves 97.92% top-1 accuracy with 16 time steps, while we obtain 98.96% with the same time steps. Compared to [26], we achieve better result with much lower latency (i.e. less than one-third of their simulation time steps) and spike-based neurons (i.e. lower energy consumption). Under the spike-based neuron setting, we outperform [26] by 3.48%. We even manage to outperform [26] under the LIAF setting with spike-based neurons. Compared to [47], we outperform them by 0.14% with the same time steps. When the time steps are enlarged to 16, we further improve the accuracy by 2.07%. Since the performance on DVS-Gesture tends to saturate, the capacity of our method is not fully demonstrated. Thus we further conduct experiments on the more challenging DVS-CIFAR10 dataset.

4.4.2. DVS-CIFAR10

Our method achieves remarkable result on DVS-CIFAR10, as shown in Table 8. Even with only 1 time step, we manage to obtain 80.3% on DVS-CIFAR10 and outperform most methods with less parameters. We even achieves better performance than [49], who uses twenty times as many time steps as ours and ten times more parameters. Although [9, 26] have less parameters than ours, we outperforms their results by a large margin with only 1 time step. Our method gains comparable results to [47,50] with much fewer time steps and parameters. These comparisons strongly support that our Spikeformer is equipped with distinguished spatio-temporal representational ability that can fully utilize limited spatio-temporal information to extract key information.

Table 10
Analysis of energy consumption. Additions/Multiplications/Memory Accesses indicate the number of additions/multiplications/memory accesses.

Model	T	Additions	Multiplications	Memory accesses	Energy efficiency
ANN counterpart	–	2.5×10^9	2.7×10^9	6.8×10^9	1×
Spikeformer-7/3 × 2 × 3	1	2.6×10^8	1.4×10^8	1.1×10^9	6.4×
	4	1.1×10^9	5.9×10^8	4.2×10^9	1.7×

4.4.3. ImageNet

To explore the potential of Spikeformer to be a large-scale SNN architecture, we conduct experiments on ImageNet dataset, as shown in Table 9. Without loss of generality, we experiment with both IF model and PLIAF model. Under the IF model setting, our Spikeformer outperforms [8] by 8.84% with less than half of their parameters and fewer time steps. With the same time steps as [9], we outperform [9] by 6.63% with fewer parameters. Furthermore, we outperform [48] by 4.65% with less than one-third of their parameters. For fair comparison, we implement an ANN counterpart under the same training strategy and our Spikeformer outperforms its ANN counterpart by 0.12%. These results indicate that Spikeformer is a promising architecture for training large-scale SNNs.

4.5. Energy consumption

In order to explore the energy-efficient potential of Spikeformer, we analyze the energy consumption of Spikeformer and its ANN counterpart on DVS-CIFAR10. Note that, for the calculation of energy consumption, we assume the data are 32-bit floating-point implementation in 45 nm technology [51], in which $E_{MAC} = 4.6pJ$ and $E_{AC} = 0.9pJ$. For SRAM memory accesses, we adopt a linear interpolation function based on 3 particular values: 8 kB($10pJ$), 32 kB ($20pJ$), 1 MB ($100pJ$) following [52]. From Table 10, we can observe that our Spikeformer shows promising potential in low-power computation. We achieve 1.7× energy efficiency over its ANN counterpart and with minor accuracy loss (1.1%), we manage to obtain 6.4× energy efficiency.

5. Discussion

In this section, we will discuss three branches of future works to fix existing limitations. First, the CT module can be further developed into a delicately designed multi-stages network (e.g. ResNet). But in this paper, we adopt simple design as not to overwhelm the importance of transformer blocks, which may be a suboptimal choice. Besides, since the ablation studies are mainly conducted on DVS-CIFAR10 and DVS-Gesture, the properties of Spikeformer on large-scale datasets are not fully demonstrated and it is necessary to explore the behavior of Spikeformer on large-scale datasets such as ImageNet. Lastly, since there are dense multiplications during self attention, a spike-based self attention mechanism can used to further make Spikeformer more energy-efficient, for example, integrating the method proposed in [21] into our method would be a feasible way since our method are orthogonal to theirs.

6. Conclusion

In this paper, we introduce spatio-temporal attention (STA) into spiking neural network (SNN) and directly train a Transformer-based SNN (i.e. Spikeformer). Replacing the original patchify stem with CT module, we manage to alleviate "data hungry", stabilize the training period and obtain a high-performance low-latency SNN. Based on extensive ablation studies, we demonstrate the cruciality of CT module and the robustness of Spikeformer in terms of depth and time steps. Furthermore, we also propose an empirical rule to design CT module (i.e. the "200 tokens" rule). Ultimately, our Spikeformer gains superior results on both neuromorphic datasets and large-scale static dataset with low latency and shows great efficiency over its ANN counterpart, which demonstrates its superior capabilities to capture spatio-temporal information and the potential to be an alternative architecture to CNN for training large-scale SNNs. We believe this architecture will bring new insight into training large-scale SNNs and promote the development of SNNs.

Funding

This research was funded by the STI 2030-Major Projects 2022ZD 0208700.

CRediT authorship contribution statement

Yudong Li: Conceptualization, Formal analysis, Investigation, Methodology, Software. **Yunlin Lei:** Conceptualization, Data curation, Methodology, Validation, Visualization. **Xu Yang:** Funding acquisition, Project administration, Resources, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Data availability

Data will be made available on request.

References

[1] Wolfgang Maass, Networks of spiking neurons: The third generation of neural network models, Neural Netw. 10 (9) (1997) 1659–1671.
[2] Michael.V. DeBole, Brian Taba, Arnon Amir, Filipp Akopyan, Alexander Andreopoulos, William.P. Risk, Jeff Kusnitz, Carlos Ortega Otero, Tapan.K Nayak, Rathinakumar Appuswamy, et al., Truenorth: Accelerating from zero to 64 million neurons in 10 years, Computer 52 (5) (2019) 20–29.
[3] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri.Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al., Loihi: A neuromorphic manycore processor with on-chip learning, Ieee Micro 38 (1) (2018) 82–99.
[4] Yangfan Hu, Huajin Tang, Gang Pan, Spiking deep residual networks, IEEE Trans. Neural Netw. Learn. Syst. (2018).
[5] Bing Han, Gopalakrishnan Srinivasan, Kaushik Roy, Rmp-snn: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020, pp. 13558–13567.
[6] Abhronil Sengupta, Yuting Ye, Robert Wang, Chiao Liu, Kaushik Roy, Going deeper in spiking neural networks: Vgg and residual architectures, Front. Neurosci. 13 (2019) 95.
[7] Emre.O Neftci, Hesham Mostafa, Friedemann Zenke, Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks, IEEE Signal Process. Mag. 36 (6) (2019) 51–63.
[8] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu, Guoqi Li, Going deeper with directly-trained larger spiking neural networks, in: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35, 2021, pp. 11062–11070.
[9] Wei Fang, Zhaofei Yu, Yanqi Chen, Tiejun Huang, Timothée Masquelier, Yonghong Tian, Deep residual learning in spiking neural networks, Adv. Neural Inf. Process. Syst. 34 (2021) 21056–21069.
[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.
[11] Karen Simonyan, Andrew Zisserman, Very deep convolutional networks for large-scale image recognition, 2015, CoRR, abs/1409.1556.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan.N Gomez, Łukasz Kaiser, Illia Polosukhin, Attention is all you need, in: Advances in neural information processing systems, vol.30, 2017.

[13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al., An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations, 2020.

[14] Tete Xiao, Mannat Singh, Eric Mintun, Trevor Darrell, Piotr Dollár, Ross Girshick, Early convolutions help transformers see better, Adv. Neural Inf. Process. Syst. 34 (2021) 30392–30400.

[15] Yahui Liu, Enver Sangineto, Wei Bi, Nicu Sebe, Bruno Lepri, Marco Nadai, Efficient training of visual transformers with small datasets, Adv. Neural Inf. Process. Syst. 34 (2021) 23818–23830.

[16] Sachin Mehta, Mohammad Rastegari, Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer, in: International Conference on Learning Representations, 2021.

[17] Seung Hoon Lee, Seunghyun Lee, Byung Cheol Song, Vision transformer for small-size datasets, 2021, arXiv preprint arXiv:2112.13492.

[18] Ali Hassani, Steven Walton, Nikhil Shah, Abulikemu Abuduweili, Jiachen Li, Humphrey Shi, Escaping the big data paradigm with compact transformers, 2021, arXiv preprint arXiv:2104.05704.

[19] Yun-Hao Cao, Hao Yu, Jianxin Wu, Training vision transformers with only 2040 images, 2022, arXiv preprint arXiv:2201.10728.

[20] Etienne Mueller, Viktor Studenyak, Daniel Auge, Alois Knoll, Spiking transformer networks: A rate coded approach for processing sequential data, in: 2021 7th International Conference on Systems and Informatics, ICSAI, IEEE, 2021, pp. 1–5.

[21] Jiqing Zhang, Bo dosng, Haiwei Zhang, Jianchuan Ding, Felix Heide, Baocai Yin, Xin Yang, Spiking transformers for event-based single object tracking, in: Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition, 2022, pp. 8801–8810.

[22] Zhou Zhaokun, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, Li Yuan, Going deeper with directly-trained larger spiking neural networks, 2022, arXiv preprint arXiv:2209.15425.

[23] Marco Cannici, Marco Ciccone, Andrea Romanoni, Matteo Matteucci, Attention mechanisms for object recognition with event-based cameras, in: 2019 IEEE Winter Conference on Applications of Computer Vision, WACV, IEEE, 2019, pp. 1127–1136.

[24] Xiurui Xie, Hong Qu, Zhang Yi, Jürgen Kurths, Efficient training of supervised spiking neural network via accurate synaptic-efficiency adjustment method, IEEE Trans. Neural Netw. Learn. Syst. 28 (6) (2016) 1411–1424.

[25] Souvik Kundu, Gourav Datta, Massoud Pedram, Peter.A. Beerel, Spike-thrift: Towards energy-efficient deep spiking neural networks by limiting spiking activity via attention-guided compression, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 2021, pp. 3953–3962.

[26] Man Yao, Huanhuan Gao, Guangshe Zhao, Dingheng Wang, Yihan Lin, Zhaoxu Yang, Guoqi Li, Temporal-wise attention spiking neural networks for event streams classification, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 10221–10230.

[27] Anthony.N. Burkitt, A review of the integrate-and-fire neuron model: I. homogeneous synaptic input, Biol. Cybern. 95 (1) (2006) 1–19.

[28] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang, Yonghong Tian, Incorporating learnable membrane time constant to enhance learning of spiking neural networks, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 2661–2671.

[29] Zhenzhi Wu, Hehui Zhang, Yihan Lin, Guoqi Li, Meng Wang, Ye Tang, Liaf-net: Leaky integrate and analog fire network for lightweight and efficient spatiotemporal information processing, IEEE Trans. Neural Netw. Learn. Syst. (2021).

[30] Friedemann Zenke, Tim.P. Vogels, The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks, Neural Comput. 33 (4) (2021) 899–925.

[31] Gedas Bertasius, Heng Wang, Lorenzo Torresani, Is space–time attention all you need for video understanding?, in: ICML. Vol. 2, 2021, p. 4.

[32] Lucas Beyer, Xiaohua Zhai, Alexander Kolesnikov, Better plain vit baselines for imagenet-1k, 2022, arXiv preprint arXiv:2205.01580.

[33] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, Mu Li, Bag of tricks for image classification with convolutional neural networks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 558–567.

[34] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey.E. Hinton, Layer normalization, 2016, arXiv preprint arXiv:1607.06450.

[35] Arnon Amir, Brian Taba, David J. Berg, Timothy Melano, Jeffrey L. McKinstry, Carmelo di Nolfo, Tapan Kumar Nayak, Alexander Andreopoulos, Guillaume Garreau, Marcela Mendoza, Jeffrey A. Kusnitz, Michael V. DeBole, Steven K. Esser, Tobi Delbrück, Myron Flickner, Dharmendra S. Modha, A low power, fully event-based gesture recognition system, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR, pp. 7388–7397.

[36] Diederik.P Kingma, Jimmy Ba, Adam: A method for stochastic optimization, in: ICLR (Poster), 2015.

[37] Li Hongmin, Hanchao Liu, Xiangyang Ji, Guoqi Li, Luping Shi, Cifar10-dvs: An event-stream dataset for object classification, Front. Neurosci. 11 (2017) 309.

[38] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, Zbigniew Wojna, Rethinking the inception architecture for computer vision, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 2818–2826.

[39] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, Li Fei-Fei, Imagenet: A large-scale hierarchical image database, in: 2009 IEEE conference on computer vision and pattern recognition, 2009, pp. 248–255.

[40] Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, Quoc V. Le, Autoaugment: Learning augmentation strategies from data, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2019.

[41] Ilya Loshchilov, Frank Hutter, SGDR: Stochastic gradient descent with warm restarts, in: International Conference on Learning Representations, 2017.

[42] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al., Mixed precision training, in: International Conference on Learning Representations, 2018.

[43] Hang Zhang, Kristin Dana, Jianping Shi, Zhongyue Zhang, Xiaogang Wang, Ambrish Tyagi, Amit Agrawal, Context encoding for semantic segmentation, in: Proceedings of the IEEE conference on Computer Vision and Pattern Recognition, 2018, pp. 7151–7160.

[44] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, Kilian.Q. Weinberger, Deep networks with stochastic depth, in: European conference on computer vision, Springer, 2016, pp. 646–661.

[45] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, in: Advances in neural information processing systems, vol.32, 2019.

[46] Fang Wei, Chen Yanqi, Ding Jianhao, Chen Ding, Yu Zhaofei, Zhou Huihui, Tian Yonghong, other contributors 2020, Spikingjelly, 2022, https://github.com/fangwei123456/spikingjelly. (Accessed: 11 June 2022).

[47] Guobin Shen, Dongcheng Zhao, Yi Zeng, Eventmix: An efficient augmentation strategy for event-based data, 2022, arXiv preprint arXiv:2205.12054.

[48] Yuhang Li, Yufei Guo, Shanghang Zhang, Shikuang Deng, Yongqing Hai, Shi Gu, Differentiable spike: Rethinking gradient-descent for training spiking neural networks, Adv. Neural Inf. Process. Syst. 34 (2021) 23426–23439.

[49] Qingyan Meng, Mingqing Xiao, Shen Yan, Yisen Wang, Zhouchen Lin, Zhi-Quan Luo, Training high-performance low-latency spiking neural networks by differentiation on spike representation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 12444–12453.

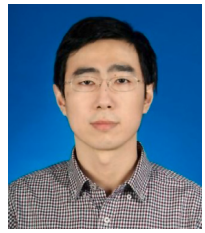
[50] Yuhang Li, Youngeun Kim, Hyoungseob Park, Tamar Geller, Priyadarshini Panda, Neuromorphic data augmentation for training spiking neural networks, 2022, arXiv preprint arXiv:2203.06145.

[51] Mark Horowitz, 1.1 Computing's energy problem (and what we can do about it), in: 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers, ISSCC, IEEE, 2014, pp. 10–14.

[52] Edgar Lemaire, Loïc Cordone, Andrea Castagnetti, Pierre-Emmanuel Novac, Jonathan Courtois, Benoît Miramond, An analytical estimation of spiking neural networks energy efficiency, in: International Conference on Neural Information Processing, Springer, 2022, pp. 574–587.

Yudong Li is currently a student in Beijing Institute of Technology.

Yunlin Lei is currently a student in Beijing Institute of Technology.



Xu Yang received the Ph.D. degree in Microelectronics from Tsinghua University, Beijing, China, in 2009. From October 2009 to February 2012, he was a postdoctoral researcher in the Department of Computer Science, Tsinghua University. Since March 2012, he has worked at Beijing Institute of Technology, Beijing, China. His research interests are in the areas of artificial intelligence, data science, and parallel computing.